

CogniFlow AI

AI-Powered Personalized Multi-Session Chatbot
with Intent Detection

Complete Project Documentation

Field	Details
Project Name	CogniFlow AI
Assignment	Round 2 Interview - Nextastra Technologies
Candidate	Dipak Raghunath Bharate
Tech Stack	Python, Streamlit, FastAPI, SQLite, Gemini AI
ML Model	TF-IDF + Logistic Regression (scikit-learn)
Image Gen	Pollinations.ai (free, no API key)

1. Project Overview

CogniFlow AI is a full-stack, AI-powered conversational chatbot built for Nextastra Technologies' Round 2 interview assignment. The system detects user intent using a custom-trained machine learning model, maintains persistent multi-session memory, generates images on demand, and produces downloadable documents — all personalized to each user.

1.1 Assignment Requirements vs Implementation

Requirement	Status	Implementation
Intent Detection	Done	TF-IDF + Logistic Regression, 91.94% accuracy
Greeting / Farewell	Done	Rule-based guardrails + ML model
General Queries	Done	Gemini API with conversation context
Contextual Conversations	Done	Last 5 messages injected into Gemini prompt
Multi-Session Memory	Done	SQLite with sessions + user_memory tables
Cross-Session Continuity	Done	user_memory table persists facts across sessions
User Preferences	Done	Gemini extracts preferences, stored in SQLite
Image Generation	Done	Pollinations.ai, intent-gated
File Generation	Done	PDF (fpdf) and DOCX (python-docx)

2. System Architecture

CogniFlow AI follows a clean client-server architecture with two independently running services:

Layer	Technology	Responsibility
Frontend	Streamlit	UI, chat display, sidebar, file downloads
Backend API	FastAPI	REST endpoints, business logic orchestration
Intent Layer	scikit-learn (TF-IDF)	Classify user input into 6 intent categories
LLM Layer	Google Gemini API	Response generation, preference extraction
Memory Layer	SQLite	Sessions, user facts, preferences
Image Layer	Pollinations.ai	Free text-to-image generation
File Layer	fpdf + python-docx	Generate downloadable PDF and DOCX files

2.1 Request Flow

- User types a message in Streamlit frontend
- Streamlit sends POST /chat to FastAPI backend
- FastAPI fetches session history and user facts from SQLite
- Intent detector classifies the message into one of 6 intents
- Based on intent: image generated, file created, or Gemini called for text response
- Gemini prompt is enriched with user preferences and cross-session facts
- Response is stored in SQLite and returned to Streamlit for display

2.2 Project File Structure

```
chatbot/  
  app/  
    main.py      # Streamlit frontend  
    api.py       # FastAPI backend (REST endpoints)  
    chatbot.py   # Core logic: intent routing, Gemini calls  
    intent_detector.py # ML model: train, evaluate, predict  
    memory.py    # SQLiteMemory class: sessions + user facts  
    image_gen.py # Pollinations.ai image generation  
    file_gen.py  # PDF and DOCX generation  
    config.py    # Environment variables (Gemini API key)  
  data/  
    intent_dataset.csv # 309 labeled training samples  
    trained_model/  
      intent_model.pkl # Saved TF-IDF + LR pipeline  
    evaluation/  
      confusion_matrix.png # Visual evaluation output  
  docs/  
    project_doc.md # Markdown documentation  
  .env             # API keys (not committed)  
  requirements.txt # All Python dependencies  
  chatbot.db       # SQLite database (auto-created)  
  test_intents.py  # Intent classifier tests
```

3. Phase 1 - Intent Detection

3.1 Model Overview

The intent detector is a scikit-learn pipeline combining TF-IDF vectorization with Logistic Regression. It classifies every user message into one of 6 intent categories before any response is generated.

Component	Configuration
Vectorizer	TfidfVectorizer — ngram_range=(1,2), lowercase=True, min_df=1
Classifier	LogisticRegression — C=10, max_iter=1000, solver=lbfgs
Pipeline	sklearn Pipeline (vectorizer -> classifier)
Saved as	intent_model.pkl (joblib)

3.2 Intent Categories

Intent	Samples	Example Utterances	Handler
greeting	51	hi, hello, good morning, hey there	Fixed response
farewell	48	bye, goodbye, take care, see you	Fixed response
image_request	50	generate an image of a dragon	Pollinations.ai
file_request	50	create a pdf report, make a word document	fpdf / python-docx
general_query	60	what is ML, tell me about Virat Kohli	Gemini API
contextual	50	do you remember what we talked about	Gemini + history

3.3 Dataset Design Decisions

- 309 total samples — balanced across all 6 intents (48-60 each)
- Removed 'ok' from farewell samples — it appeared in introductions causing misclassification
- Added 'ok' to general_query samples — teaches model 'ok' is a neutral word
- Added introduction phrases to greeting — 'my name is', 'my self is', 'myself is'
- Added 15 person-info queries to general_query — 'tell me about APJ Abdul Kalam'
- Bigrams (ngram_range=1,2) help distinguish 'image request' vs 'file request' phrases

3.4 Rule-Based Guardrails

In addition to the ML model, detect_intent() includes hard-coded guardrails to prevent known failure cases:

```
# Guardrail 1: introduction phrases always return greeting
intro_signals = ['my name is', 'my self is', 'myself is', 'call me', ...]
for signal in intro_signals:
    if signal in text: return 'greeting'
```

```
# Guardrail 2: clear farewell words with no greeting words force farewell
has_farewell = any(s in text for s in ['bye','goodbye','see you',...])
has_greeting = any(s in text for s in ['hi','hello','hey',...])
if has_farewell and not has_greeting: return 'farewell'

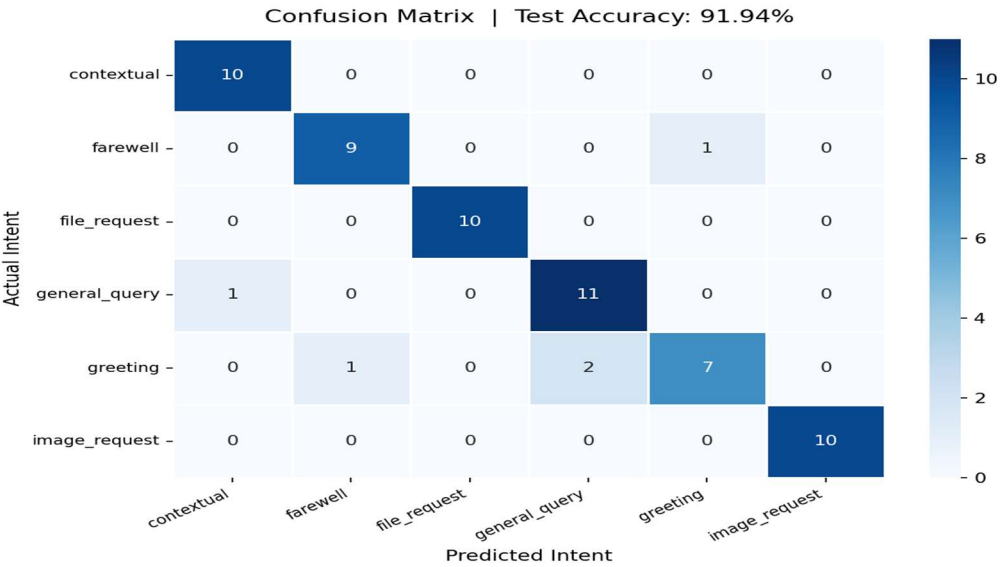
# All other inputs go to the ML model
return classifier.predict([user_input])[0]
```

3.5 Training & Evaluation Results

Metric	Value
Total Dataset Size	309 samples
Training Samples	247 (80%)
Test Samples	62 (20%)
Test Set Accuracy	91.94%
5-Fold CV Mean Accuracy	90.90%
5-Fold CV Std Dev	0.0723

3.6 Per-Class Classification Report

Intent	Precision	Recall	F1-Score	Support
contextual	0.91	1.00	0.95	10
farewell	0.90	0.90	0.90	10
file_request	1.00	1.00	1.00	10
general_query	0.85	0.92	0.88	12
greeting	0.88	0.70	0.78	10
image_request	1.00	1.00	1.00	10
Weighted Avg	0.92	0.92	0.92	62



4. Phase 2 - Multi-Session Memory

4.1 Database Design

Memory is managed by the SQLiteMemory class using three tables in a local SQLite database (chatbot.db):

Table	Primary Key	Purpose
sessions	session_id	Stores full conversation history per chat session as JSON
user_profiles	username	Stores learned user preferences as JSON
user_memory	(user_id, key)	Stores permanent facts about user across ALL sessions

4.2 Cross-Session Continuity

The user_memory table is the key to cross-session memory. Unlike sessions (which are isolated), user_memory persists across every new chat:

```
# Facts saved after every message
if 'my name is' in user_input.lower():
    name = user_input.split('my name is')[-1].strip()
    memory.save_user_fact(username, 'name', name)
memory.save_user_fact(username, 'last_topic', user_input[:100])

# Facts injected into Gemini prompt for every new session
facts = memory.get_user_facts(username)
memory_context = f"""
You already know this about the user from previous sessions:
- Name: {facts.get('name', 'unknown')}
- Last topic discussed: {facts.get('last_topic', 'none')}
"""
```

4.3 Preference Learning

After every user message, Gemini extracts preferences and stores them in user_profiles. These are injected into subsequent prompts for personalization:

- Chatbot.learn_preferences() calls Gemini to extract implied preferences from user messages
- Preferences are stored as JSON — e.g. {"vehicle": "Toyota Fortuner", "city": "Pune"}
- Every Gemini prompt includes current preferences for personalized responses
- Preferences accumulate over time — each session adds to the profile

4.4 Session Management

- Each chat session has a unique UUID as session_id
- New Chat button creates a new session_id — fresh conversation starts
- Old sessions are listed in the sidebar with their first message as title
- Sessions can be deleted individually via the trash icon
- Cross-session memory (user_memory) is NOT cleared when a session is deleted

5. Response Generation

5.1 Gemini Integration

General queries and contextual conversations are handled by Google Gemini 1.5 Flash Lite via the google-genai SDK. The prompt is constructed dynamically with all available context:

```
prompt = f"""
You already know this about the user from previous sessions:
- Name: {facts.get('name')}
- Last topic: {facts.get('last_topic')}

You are a helpful and personalized AI assistant.
User Profile & Preferences: {json.dumps(user_prefs)}

Conversation history (last 5 messages):
{formatted_history}

User's current query: {message}
Please respond accordingly.
"""
```

5.2 Image Generation

Images are generated using Pollinations.ai — a free, no-signup image generation API. The image is downloaded server-side and served as a local file to avoid browser CORS issues:

```
url = f'https://image.pollinations.ai/prompt/{encoded_prompt}?seed={seed}&width=1024&height=1024'
response = requests.get(url, headers={'User-Agent': 'Mozilla/5.0 ...'})
filepath = f'generated_img_{seed}.jpg'
# Image saved locally and displayed via st.image(filepath)
```

5.3 File Generation

Format	Library	How it Works
PDF	fpdf2	Gemini generates content text, fpdf writes it page-by-page with latin-1 encoding
DOCX	python-docx	Gemini generates content text, python-docx wraps it in a Word document

6. API Reference (FastAPI)

Method	Endpoint	Description
POST	/chat	Main chat endpoint — processes message, returns response
GET	/history/{session_id}	Returns full conversation history for a session
GET	/sessions/{username}	Returns all sessions for a user
DELETE	/history/{session_id}	Deletes a session and its history
GET	/profile/{username}	Returns learned preferences for a user

6.1 Chat Request Schema

```
{
  "session_id": "uuid-string",    // current session ID
  "message":   "user input text", // user message
  "username":  "dipak",           // identifies the user
  "file_format": "pdf"           // "pdf" or "docx"
}
```

6.2 Chat Response Schema

```
// Text response
{ "type": "text", "content": "Hello! How can I help you?" }

// Image response
{ "type": "image", "content": "Here is your image:", "url": "/path/to/image.jpg" }

// File response
{ "type": "file", "content": "PDF generated.", "filepath": "...", "filename": "..." }
```

7. Setup & Running the Project

7.1 Prerequisites

- Python 3.10 or higher
- Google Gemini API key (free at aistudio.google.com)
- All dependencies from requirements.txt

7.2 Installation

```
# Step 1: Install dependencies
pip install -r requirements.txt

# Step 2: Create .env file in project root
GEMINI_API_KEY=your_gemini_api_key_here

# Step 3: Train the intent model (first time only)
python app/intent_detector.py
```

7.3 Running the Application

```
# Terminal 1: Start FastAPI backend
uvicorn app.api:app --reload --port 8001

# Terminal 2: Start Streamlit frontend
streamlit run app/main.py

# Open browser at: http://localhost:8501
```

8. Dependencies

Package	Version	Purpose
streamlit	latest	Frontend UI
fastapi	latest	REST API backend
uvicorn	latest	ASGI server for FastAPI
google-genai	latest	Gemini AI response generation
scikit-learn	latest	TF-IDF + Logistic Regression
joblib	latest	Model serialization (.pkl)
pandas	latest	Dataset loading and processing
matplotlib	latest	Confusion matrix chart
seaborn	latest	Confusion matrix styling
numpy	latest	Numeric operations
fpdf2	latest	PDF generation
python-docx	latest	DOCX generation
requests	latest	Image downloading from Pollinations
python-dotenv	latest	Load .env environment variables
pydantic	latest	Request/response schema validation

9. Key Design Decisions

Decision	Rationale
TF-IDF over LSTM	Faster training, no GPU needed, 92% accuracy sufficient for 6-class problem
Rule-based guardrails	Prevents known edge cases (introductions misclassified as farewell)
SQLite over cloud DB	Zero setup, runs locally, sufficient for single-user demo
FastAPI + Streamlit split	Clean separation of concerns; API can be tested independently
Pollinations.ai for images	Completely free, no API key required, generates 1024x1024 images
Server-side image download	Avoids browser CORS issues when rendering images in Streamlit
Gemini for preference learning	Avoids hardcoded regex; LLM naturally extracts rich preferences
Last 5 messages in context	Balances memory depth with Gemini token limits

10. Results Screenshot

